

Math/ML library with AI Applications

Authors: Alex Kozik (ajk333), Nick Brenner (nlb74), Kevin Weng Jr. (kw444)

Course: CS 4701 - Spring 2025

AI Keywords

Supervised Learning

- Perceptron
- Linear Regression
- Logistic Regression
- Support-Vector Machines (SVMs)
- Unsupervised Learning
 - Unsupervised learning will be used for the k-Means clustering algorithm that will categorize sheet music from classical composers by era (Romantic, Baroque, Classical, etc.).
- Reinforcement Learning
 - Multi-Arm Bandits (MABs) will use the above clustering algorithm to identify the "Arms" and use various MAB algorithms (epsilon greedy, upper confidence bound, etc.) to explore how to best recommend classical music.
 - Time permitting, we will try to implement Bandit-based Clustering with Exploration (BCC) to use RL to fine-tune the clustering.
- Optimization
 - We aim to implement the above algorithms from scratch using C++ and to compare our implementations accuracy and efficiency with that of Scikit-Learn, Tensorflow, PyBandit

Application Setting

General machine learning and artificial intelligence.

Description

What we want to do:

First, we will implement a low-level ML library in C++. This involves the following components:

Matrix Operations

- Matrix multiplication
- Matrix inversion
- Transposition
- Eigenvalue/eigenvector computation using iterative QR algorithm
- Scalar multiplication

Vector operations

- Basic arithmetic and inner products

Numerical differentiation

- Approximating derivatives for optimization purposes

Expression Trees

- Abstract Syntax Tree (AST) representation for mathematical expressions
- Expression evaluation engine
 - Used for expressing/evaluating loss function in ML models
- Lexer, parser, and interpreter for the grammar

Using these components, we implement the models outlined in "AI Keywords" (see above).

Using this library, our projection will have three aims:

1. Testing Binary Classification
 - Logistic regression
 - Support Vector Machine
 - Perceptron
2. Testing linear regression.
3. Testing Unsupervised Learning/RL.

Aspects of AI:

Our project will work with the following aspects of AI:

1. Binary Classification

Multiple of our implemented models exist in binary classification settings, namely Perceptron, Logistic Regression, and SVMs. To apply these, we would choose from the following application ideas:

1. Classifying music between instrumental and synthesized
2. Classifying music between happy or sad
3. Classifying music as major or minor

Note if the above prove untenable, we will pursue other tasks such as:

1. Spam email classification
2. Sentiment analysis
3. Disease diagnosis
4. Malicious URL classification

2. Linear regression

For linear regression, we need a regression setting. Therefore, we will choose to apply our implementation to the following tasks:

1. Predicting music duration based on its characteristics

2. Predicting song popularity based on its characteristics

If any of the above prove untenable, we will pursue other tasks such as:

1. Stock market prediction
2. Real estate predictor
3. Traffic flow prediction
4. Temperature prediction

3. Unsupervised Learning/Reinforcement Learning

To apply our unsupervised (clustering) algorithm and reinforcement learning (MAB), we propose the following idea:

There are tons of features in classical music around which pieces may be categorized: Genre, Era, Composer, etc. But do these necessarily correspond to what is most interesting for a user? For example, a user may love the lavish melodies of Tchaikovsky coupled with the rhythmic precision of the Baroque Era.

Our end goal is to come up with an AI solution for music recommendation for classical instrumentalists. But we propose doing this in a (unique) way in hopes of better results.

Due to the aforementioned problem of classical categories not necessarily corresponding to user preferences, we first seek to use a clustering algorithm like k-Means to identify relationships within the dataset of classical music. We will fine-tune the clustering algorithm and the parameters as needed.

We take these clusterings as the proposed "arms" for a Multi-Armed Bandit. The Bandit will have these initial clusterings to choose from.

Next, the user can listen to the proposed song and react with some prescribed sentiment, indicating their preference.

We will explore various MAB algorithms (epsilon greedy, UCB, etc.) to optimize the bandits actions.

Time permitting, we would then like to implement Bandit-based Clustering with Exploration (BCC). Many papers have proposed algorithms for doing this, but there is not one universal state-of-the-art choice. Hence, we would first try a "weighted-means" approach, where instead of just the raw means in k-Means, we would weight the contribution of each datapoint by the associated feedback from the user.

This will create a feedback loop for clustering and hopefully provide for tailored categories for a particular user.

How we plan to evaluate:

Most of our models have built-in metrics for evaluation (supervised learning models). We will train the models on specific datasets and then gauge their accuracy. That will be our metric for how effective they are. We will also compare these metrics with Scikit-Learn/PyTorch/Tensorflow implementations of the corresponding model for both accuracy and efficiency (how long the models take to train).

The same can be done in the RL setting, namely comparing against the corresponding MAB in PyBandit.

Timeline

- Select datasets for the above tasks **due 3/16**
- Implement models in Python **due 3/28**
 - Completing the above project in Python
 - Aggregate results for later comparison
- Finalize ML components in C++ **due 4/12**
- Finalize ML models in C++ **due 4/19**
- Adjust Python implementations to use implemented ML library **due 4/26**

Existing Resources

Software resources:

- Python
- C++
- Scikit-Learn
- PyBandit
- Numpy

Data resources:

TBD

References

N/A